Sardar Vallabhbhai National Institute of Technology
Department of Artificial Intelligence (B.Tech. III)

Subject: Robotics and its Application AI352

Lab 7: Inverse Dynamics Control & Simulation

INSTRUCTION:
1. Please save your lab.doc as LAB_No_Roll_No.doc.
2. Use/paste the snapshot of the steps followed along with result/s.
3. Mention your observation/comment after results in the doc.
4. Any violation from the Academic honesty, will results in the strongest consequences available to us.
5. Late Policy: 10% deduction per day

Part A: Lagrange- Euler Inverse Dynamics

Objective: To calculate time-varying torques for a 2-DOF planar robot following a smooth trajectory and animate the results using Python.

1: A 2-DOF planar robot must move its end-effector smoothly from a resting horizontal position to a vertical position in 5 seconds.
Robot Parameters:

- Lengths: L_1 = 1.0 m, L_2 = 1.0 m
- Masses: m_1 = 4.0 Kg, m_2 = 2.0 kg (treated as point masses at the ends of links)
- Friction: fc = 1.5 Nm(Coulomb), v = 0.3\text Nm s/rad (Viscous)
- Gravity: g = 9.81m/s^2

Desired Trajectory:

The joints follow a "Quintic Polynomial" or "Smooth Sine" profile to ensure acceleration starts and ends at zero:

- $\theta_1(t) = \frac{\pi}{2}(1-\cos(\frac{\pi t}{5}))$
- $\theta_2(t) = \frac{\pi}{4}\sin(\frac{\pi t}{5})$

2: Manual Calculation (Checkpoint)

Before coding, calculate the torque τ_1 *and* τ_2 *at* $t = 2.5$ seconds.

1. Determine the instantaneous positions (θ), *velocities*$(\dot{\theta})$, *and accelerations*$(\ddot{\theta})$*at* $t = 2.5$.
2. Use the Manipulator Equation to find the torque:

$$\tau = M(\theta)\ddot{\theta} + V(\theta,\dot{\theta}) + G(\theta) + \tau_{friction}$$

3: Python Implementation & Animation

Write a Python script to perform the following:

1. Define the Dynamics Function: Create a function that takes $\theta, \dot{\theta}, \ddot{\theta}$ as inputs and returns τ_1, τ_2
2. Time-Series Generation: Calculate the torque for 500 time steps between $0 \le t \le 5$
   Visualization:
   - Plot 1: Create a 2D animation of the robot arm moving.
   - Plot 2: Create a live-updating line chart showing the Total Torque vs. Time for both joints.
   - Plot 3: Show the individual contribution of the Gravity term versus the Inertia term.

4: Lab Report Questions

1. Peak Torque: At what point in the trajectory was the torque at Joint 1 the highest? Was this due to gravity or acceleration?
2. Negative Torque: Did you encounter any negative torque values? Explain why a motor would need to provide "negative" torque during a smooth upward lift.
3. Coupling Analysis: Change the acceleration of Joint 2 $\ddot{\theta}_2$ to be very high. How does this affect the torque required at Joint 1, even if Joint 1 is stationary?

5: Bonus Challenge: Add a payload of 2 kg to the end-effector. Modify your script to show how the "Inertia Matrix" $M(\theta)$ changes its values when the robot is fully extended versus when it is tucked in.

Part B: Lagrange- Euler Inverse Dynamics

This exercise focuses on the inverse-dynamics analysis of a planar 2-DOF (RR) robot manipulator. You will implement a resolved-rate control scheme to track a Cartesian velocity and calculate the required torques using the equations of motion.

1. Robot Parameters & Setup

The robot consists of two links of lengths L_1 and L_2. Both links are solid steel rectangular prisms.

- Geometry: $L_1 = 1.0 \text{ m}, L_2 = 0.5 \text{ m}$
- Cross-section: Width $w = 0.05 \text{ m}$ Thickness $t = 0.05 \text{ m}$
- Material: Steel density $\rho = 7806 \text{ kg/m}^3$
- Initial State: $\Theta_0 = \begin{bmatrix} 10^\circ \\ 90^\circ \end{bmatrix}$ (Convert these to radians for calculation).

Note: Calculate the mass m_i and the moment of inertia $I_{zz,i}$ for each link relative to its center of mass before starting the simulation.

2. Control Objective

The robot must move at a constant Cartesian velocity relative to the base $frame\{0\}: {}^{0}\dot{X} = \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$ m/s

3. Simulation Requirements

Run the simulation for 1.0 second with a time step of $\Delta t = 0.01$ s

At each step:

1. Kinematics: Compute the Jacobian $J(\theta)$
2. Resolved-Rate: Find joint velocities using $\dot{\Theta} = J^{-1} \cdot {}^{0}\dot{X}$
3. Numerical Differentiation: Calculate joint accelerations $\ddot{\Theta}$ from $\dot{\Theta}$
4. Inverse Dynamics: Calculate the required torques τ using the manipulator equation:

$$\tau = M(\Theta)\ddot{\Theta} + V(\Theta, \dot{\Theta}) + G(\Theta)$$

4. Tasks & Visualization

Perform the simulation twice:

- Case A: Ignore gravity (g = 0).
- Case B: Include gravity (g = 9.81m/s^2 in the negative Y direction).

Generate the following five plots using matplotlib:

1. Joint Angles: θ_1, θ_2 (degrees) vs. time.
2. Joint Rates: $\dot{\theta}_1, \dot{\theta}_2 (rad/s)$ vs. time.
3. Joint Accelerations: $\ddot{\theta}_1, \ddot{\theta}_2 (rad/s^2)$ vs. time.
4. End-Effector Path: x vs. y (Cartesian space).
5. Joint Torques: τ_1, τ_2 (Nm) vs. time.

To complete this, I recommend using the following imports:

Python

```
import numpy as np
import matplotlib.pyplot as plt
from numpy.linalg import inv
```